

Event Collision Detector: Predictive Launch-Window Intelligence

Project Proposal for UCS503P

Thapar Institute of Engineering and Technology

Student Names	Jaskaran Singh, Vaani Mehta, Parmeet Singh
Roll Numbers	1024160033, 1024160123, 1024160021
Course	Software Engineering (UCS503P)
Submitted to	Jeelani Asif

1 Higher-order goal

The project aims to improve the success of high-stakes launches by reducing avoidable competition for audience attention, media coverage, and marketing visibility. The immediate engineering objective of the **Event Collision Detector** is to convert cross-industry event intelligence into an actionable scheduling decision by aggregating external events, evaluating a proposed launch window, generating an explainable collision-risk score, and recommending lower-risk alternatives.

2 Time-to-value

- Deliver an initial single-industry event ingestion pipeline and heuristic collision score within the first iteration.
- Prioritize fast feedback using existing event APIs and explainable rule-based scoring before introducing advanced predictive models.
- Define first-iteration success as a complete date input → event retrieval → risk score → recommendation flow.

3 Problem Statement

Marketing and product teams often choose launch dates using internal schedules without sufficient visibility into the external event landscape.

- **Blind-Spot Scheduling:** Internal calendars do not show competitor launches, industry conferences, or major public events.
- **Diluted Audience Attention:** Overlapping events fragment media coverage and audience engagement.
- **Sunk Marketing Spend:** A poorly timed launch can reduce the return on months of campaign preparation.

- **No Systematic Alternative-Date Discovery:** Teams lack a structured way to identify nearby low-risk windows.

4 Proposed Solution

4.1 Overview

Build a web-based **Event Collision Detector** with the following components:

- **Cross-Industry Event Radar:** Aggregates external events from APIs, conference schedules, and selected public sources into an indexed event catalog (D2: **Event Index**).
- **Real-Time Collision Scoring:** Produces a High / Medium / Low risk score for a proposed date by searching nearby dates (± 7 days).
- **Optimal Launch Recommendations:** Searches surrounding dates and returns lower-risk alternatives.
- **Interactive Risk Dashboard:** Displays the risk score, conflicting events, and explanations.
- **Calendar Syncing & Source Management:** Allows users to sync finalized dates to Google/Outlook calendars and enables administrators to manage industry configurations and users.

4.2 Core workflow

1. **Ingest & Normalize:** Ingestion jobs continuously normalize raw event data into D2: **Event Index** (Elasticsearch) based on admin industry rules.
2. **Process Submission:** User submits launch date details; validated parameters and user session records are logged to D1: **User Store** (PostgreSQL).
3. **Retrieve Events:** System queries D2 for competing events occurring within ± 7 days of the target date.
4. **Compute Risk & Recommendations:** Scoring engine computes collision risk, identifies primary drivers, and identifies low-risk alternative launch windows.
5. **Render Dashboard:** Interactive dashboard renders collision risk scores, event breakdowns, and recommended date cards.
6. **Sync Confirmed Date:** User selects a final date, updating D1 and pushing an OAuth event payload to external calendar platforms (Google/Outlook).



Figure 1: Event Collision Detector Core Workflow Execution Pipeline (Aligned with Level 1 DFD)

4.3 Operational constraints for an educational setting

- Limit initial event coverage to selected industries such as Technology and Entertainment.
- Prefer existing APIs and structured public feeds over large-scale scraping infrastructure.
- Use heuristic, explainable scoring so that each risk score can be traced to specific events.
- Keep the system deployable on standard cloud infrastructure or containerized local environments.

5 System Analysis & Behavioral Diagrams

5.1 Level 1 Data Flow Diagram (DFD)

The Level 1 DFD models the flow of data across the system entities, processes, and persistent data stores.

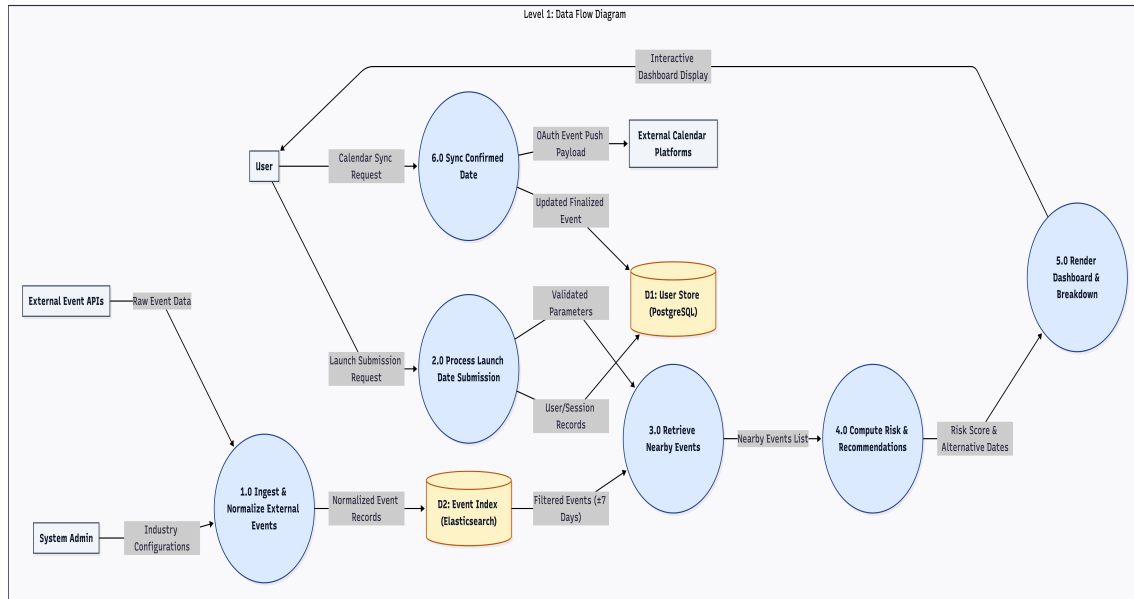


Figure 2: Level 1 Data Flow Diagram (DFD) for Event Collision Detector

5.2 Use Case Diagram

The Use Case Diagram defines the interactions between key system actors and the primary functional features.

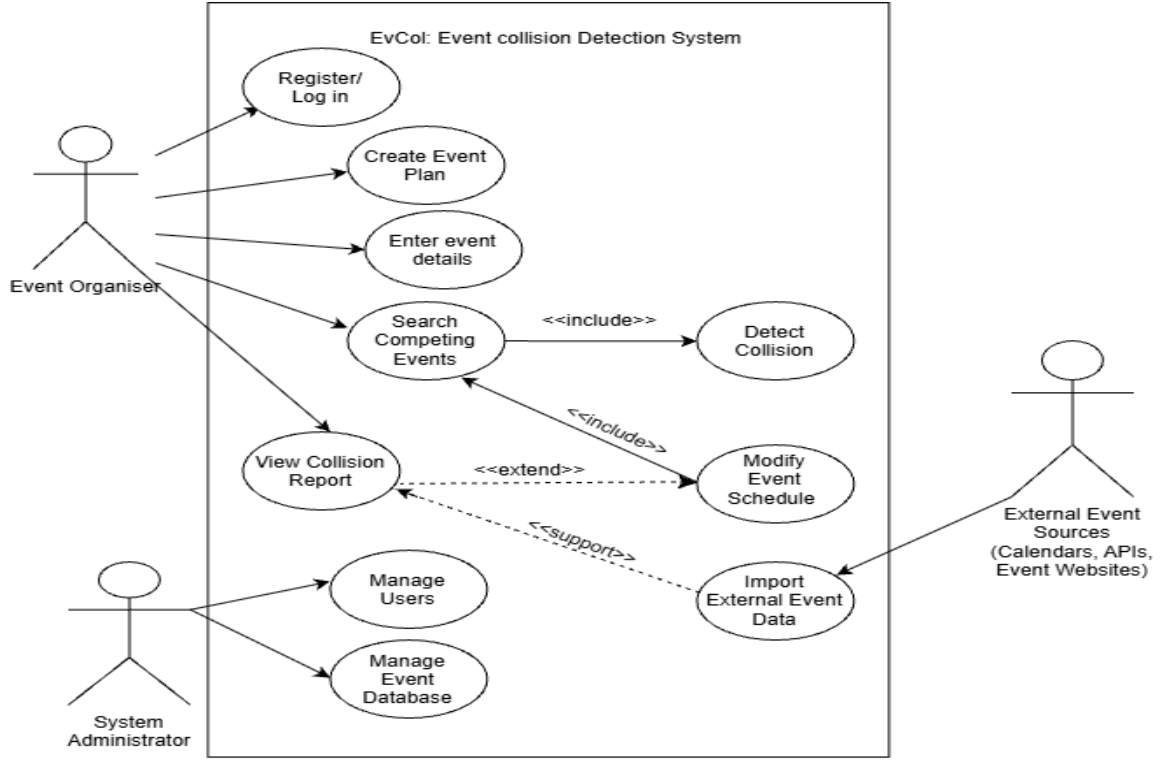


Figure 3: System Use Case Diagram with Actors and Boundary Constraints

6 Solution Approach

This project focuses on software engineering, event ingestion, low-latency retrieval, explainable scoring, and a responsive dashboard rather than novel machine-learning research.

6.1 Collision analysis logic

- **Temporal Proximity:** Events closer to the proposed date (± 7 day window) receive greater weight.
- **Industry Relevance:** Same-industry or related events contribute more strongly to risk.
- **Audience Overlap:** Similar target-audience tags increase attention conflict.
- **Event Magnitude:** Larger events receive greater importance.
- **Media Friction:** Events competing for similar channels or press attention increase the score.



Figure 4: Collision Analysis and Explainable Risk-Scoring Logic

6.2 System architecture

A layered architecture separates user interaction, application logic, scoring, ingestion, and persistent storage.

- **Frontend Dashboard:** React.js + Tailwind CSS for rendering the interactive dashboard and calendar sync actions.
- **API Layer:** FastAPI backend handling submission processing, date retrieval, and OAuth calendar synchronization.
- **Collision Scoring Service:** Python analysis service executing proximity and audience collision heuristics.
- **Data Ingestion Layer:** Ingestion service that retrieves raw data from External Event APIs and normalizes records into Elasticsearch.
- **Storage Layer:** Dual-store database setup featuring D1: **User Store** (PostgreSQL) for user/session records and confirmed events, and D2: **Event Index** (Elasticsearch) for fast ± 7 day date-range queries.

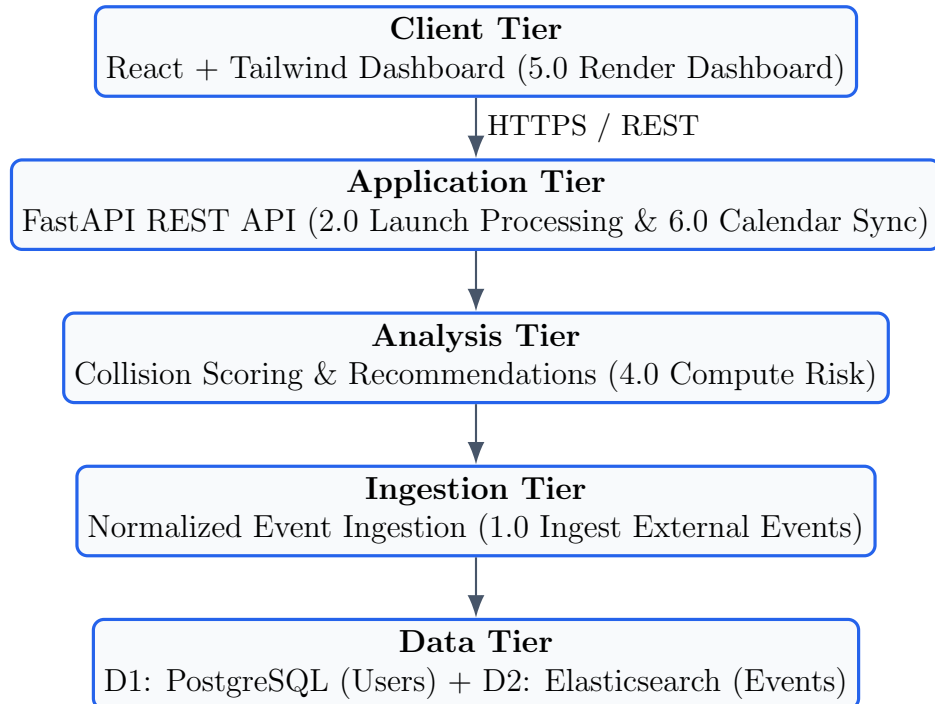


Figure 5: Event Collision Detector Layered Web Architecture

6.3 CI/CD and fast delivery enabler

CI/CD pipelines will run unit tests for scoring and normalization logic, verify frontend-backend API contracts, validate builds, and publish project documentation through GitHub Actions.

7 Evaluation Criterion

7.1 Primary evaluation metric

Collision Score Latency (CSL): Time from proposed-event submission to returned risk score and recommendation set.

- **Target:** Median CSL ≤ 500 ms against an indexed Elasticsearch event dataset (D2).
- **Measurement:** Backend request timing recorded for each scoring request.

7.2 Secondary evaluation metrics

- **Scoring Agreement:** At least 80% agreement with manual assessment on a labeled validation set.
- **Ingestion Freshness:** Newly published API events reflected in D2 within 24 hours of the ingestion cycle.
- **Recommendation Quality:** At least 90% of recommended dates should have a lower score than the original date.
- **Dashboard Responsiveness:** Risk details and recommendations render on the interactive dashboard within one second after the API response.

7.3 Pilot validation plan

Test historical launch dates against manually researched competing events and conduct a small usability trial with students role-playing marketing or product managers.

8 Scalability

1. **Scalable theoretically:** Separate ingestion (1.0) from scoring (4.0), cache repeated date windows in Elasticsearch (D2), and introduce query optimization as event volume grows.
2. **Operationally deployable practically:** Containerize frontend and backend, add industries through new source configurations, and run initially on PostgreSQL (D1) and Elasticsearch (D2).

9 Engine availability heuristic

- **React + Tailwind CSS:** Frontend dashboard and interactive rendering.
- **FastAPI:** Python backend for event, scoring, and calendar sync endpoints.
- **PostgreSQL (D1):** Storage for user accounts, session records, and updated finalized events.
- **Elasticsearch (D2):** Index for normalized external events and low-latency date-range filtering.
- **Pandas / Scikit-learn:** Data preprocessing and scoring utilities.
- **External Event APIs / OAuth:** Ingestion sources (Eventbrite/PredictHQ) and calendar push platforms (Google/Outlook).
- **GitHub Actions:** Automated testing and CI/CD.

10 Project scope and deliverables

10.1 Initial deliverable

- Basic React dashboard for event submission and rendering.
- One external event source integrated into the ingestion pipeline (1.0).
- Working FastAPI endpoint returning High / Medium / Low collision risk (4.0).
- Risk breakdown showing the events responsible for the score.
- PostgreSQL (D1) and Elasticsearch (D2) persistence setup.
- Automated CI pipeline for linting, testing, and build validation.

10.2 Subsequent deliverables

- Additional event sources and industry coverage.
- Alternative-date recommendation engine.
- OAuth calendar synchronization for Google and Outlook platforms (6.0).
- Administrator interface for user and event database management.

11 Risks and mitigations

- **API Rate Limits / Data Gaps:** Cache retrieved events in D2 and reduce confidence when source data is incomplete.
- **Scoring Heuristic Inaccuracy:** Validate weights against manually labeled historical examples.
- **Cold-Start Coverage:** Seed the Elasticsearch index with an initial historical batch.
- **Inconsistent Event Metadata:** Normalize incoming events into a common schema during Process 1.0 before scoring.
- **Over-Engineering:** Keep the first version limited to one or two industries and an explainable scoring function.

12 Summary

The **Event Collision Detector** is a deployable scheduling-intelligence platform that helps teams avoid launching into crowded attention windows. It combines external event aggregation, explainable collision scoring, and alternative-date recommendations in a single web dashboard. The project emphasizes rapid time-to-value, measurable evaluation criteria, modular architecture, mature tooling, and a clear path from prototype to scalable product.